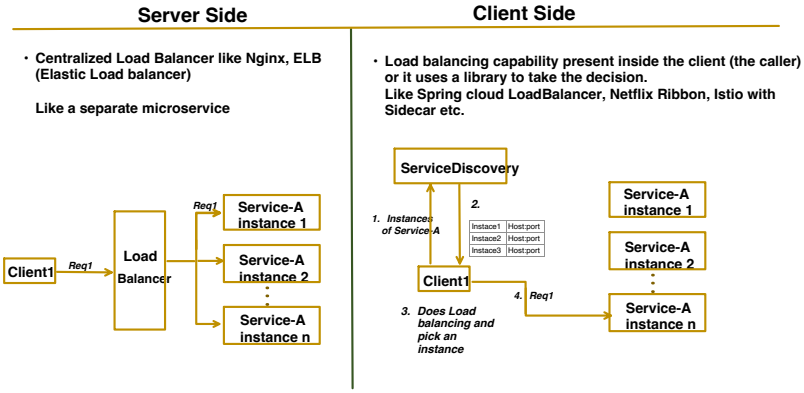
Client Side Load Balancer

Wednesday, 18 June 2025 3:37PM

Load Balancer:

- Helps in distributing traffic to multiple instances of a servers.
- Helps in preventing single server from being overloaded with huge traffic.

Load Balancer Types:



Client Side Load Balancer:

Pre-requisite :

Let's see Client Side Load Balancing with RestTemplate:

In Service Discovery topic, we have seen that:

- After setting up the Service Discovery.
- In RestTemplate, we fetch the List of Instances using DiscoveryClient object.
- Then client is manually choosing the instances from the list and invoking the endpoint.

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    DiscoveryClient discoveryClient;

    @Autowired
    RestTemplate restTemplate;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        List<ServiceInstance> instances = discoveryClient.getInstances( serviceId: "product-service");
        URI uri = instances.get(0).getUri();

        String response = restTemplate.getForObject( url: uri + "/products/" + id, String.class);

        System.out.println("Response from Product api call is: " + response);
    }
}
```

```java
@Configuration
public class Config {

    @Bean
    public RestTemplate restTemplateObj() {
        return new RestTemplate();
    }
}
```

Now, lets use "Spring Cloud Load Balancer" and see the above example again:

1. Add dependency

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
```

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestTemplate restTemplate;

    @Value("${product.service.baseurl}")
    String productBaseURL;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        String response = restTemplate.getForObject( url: productBaseURL + "/products/" + id, String.class);
        System.out.println("Response from Product api call is: " + response);
    }
}
```

Now, we don't need DiscoveryClient here.

```java
@Configuration
public class Config {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplateObj() {
        return new RestTemplate();
    }
}
```

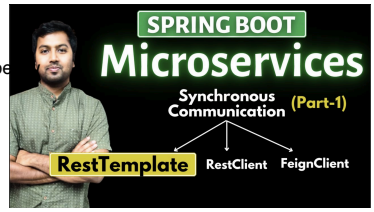
With this annotation framework start intercepting the request, call Service Discovery, get instances & apply load balancing algorithm.

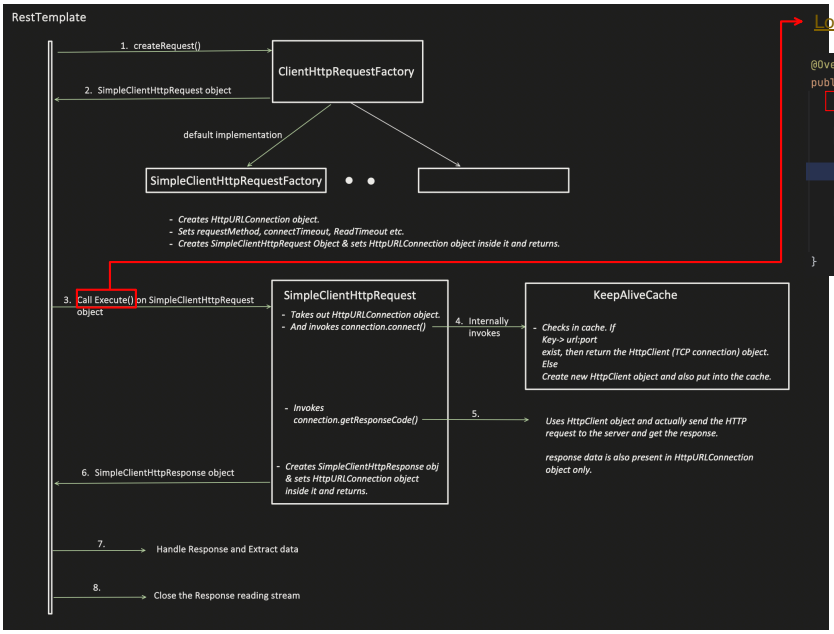
application.properties

```
server.port=8081

spring.application.name=order-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

product.service.baseurl = http://product-service
```

Now we are simply using the product service name.

In RestTemplate topic, we have already seen be
diagram and understood it in-depth:





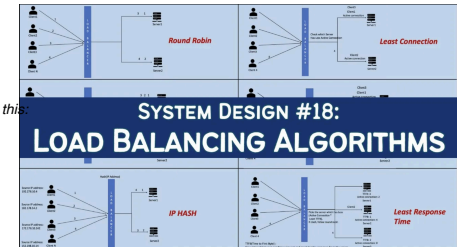
LoadBalancerInterceptor.java class get invokes first

```java
@Override
public <T> ServiceInstance choose(String serviceId, Request<T> request) {
    ReactiveLoadBalancer<ServiceInstance> loadBalancer = loadBalancerClientFactory.getInstance(serviceId);
    if (loadBalancer == null) {
        return null;
    }
    Response<ServiceInstance> loadBalancerResponse = Mono.from(loadBalancer.choose(request)).block();
    if (loadBalancerResponse == null) {
        return null;
    }
    return loadBalancerResponse.getServer();
}
```

1. Picks the load balancing algorithm

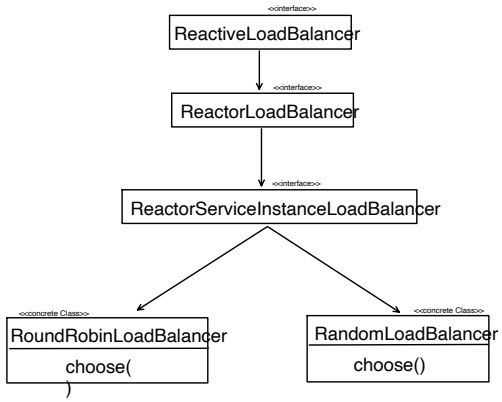
2. Internally calls Service Discovery and picks one instance based on above load balancing algorithm.

In HLD playlist, I have already discussed different types of Load balancer algorithm in depth, have a look, if there is any doubt with this.



`ReactiveLoadBalancer<ServiceInstance> loadBalancer = loadBalancerClientFactory.getInstance(serviceId);` selects load balancer algorithm for specific SERVICE ID

- Each Load Balancing Algorithm is attached to a ServiceId. ServiceId is nothing but a client name like : "*product-service*".
- By default Spring Cloud Load Balancer picks RoundRobinLoadBalancer algorithm.
- Means if we are not going with Default Algorithm, then we need to define for which client (or serviceId) which algorithm we need.
- Spring Cloud Load Balancer supports only 2 algorithms i.e. RoundRobin and Random.
- If we want different algorithm like Weighted, Least Connection etc. then either we need to write custom implementation or switch to service mesh like Istio with Side car.



If we want to use say RandomLoadBalancer instead of Default RoundRobinLoadBalancer, we can override it in configuration

(Same as previous)

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestTemplate restTemplate;

    @Value("${product.service.baseurl}")
    String productBaseURL;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        String response = restTemplate.getForObject( url: productBaseURL + "/products/" + id, String.class);
        System.out.println("Response from Product api call is: " + response);
    }
}
```

(Same as previous)

```java
@Configuration
public class Config {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplateObj() {
        return new RestTemplate();
    }
}
```

application.properties (Same as previous)

```
server.port=8081

spring.application.name=order-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

product.service.baseurl = http://product-service
```

Now Changing the Load Balancing algorithm for Product Service from default RoundRobinLoadBalancer to RandomLoadBalancer.

```java
@SpringBootApplication
[redacted]
@LoadBalancerClient(name = "product-service", configuration = LoadBalancerProductClientConfig.class)
public class OrderserviceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}
```

Normal Config class(like above) generally invokes only at application startup.
But this LoadBalancerClient config invokes at runtime too, when "product-service" is used: "http://product-service/xyz"

```java
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new RandomLoadBalancer(
                factory.getLazyProvider( name: "product-service", ServiceInstanceListSupplier.class),
                serviceId: "product-service");
    }
}
```

ServiceInstanceListSupplier, is actually the one, which helps to invoke Service Discovery and ask for instances of "product-service"

New RandomLoadBalancer(ServiceInstanceListSupplier, serviceId)
we pass serviceId because it mapped this algorithm to this service ID only.
So for "product-service" only RandomLoadBalancer is used.

What if, for "product-service" I want Algorithm A and for all other clients (or ServiceId), I want Algorithm B

```java
@SpringBootApplication
[redacted]
@LoadBalancerClients(defaultConfiguration= LoadBalancerGlobalConfig.class,
        value = {
        @LoadBalancerClient(name = "product-service", configuration = LoadBalancerProductClientConfig.class)
        })
public class OrderserviceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}
```

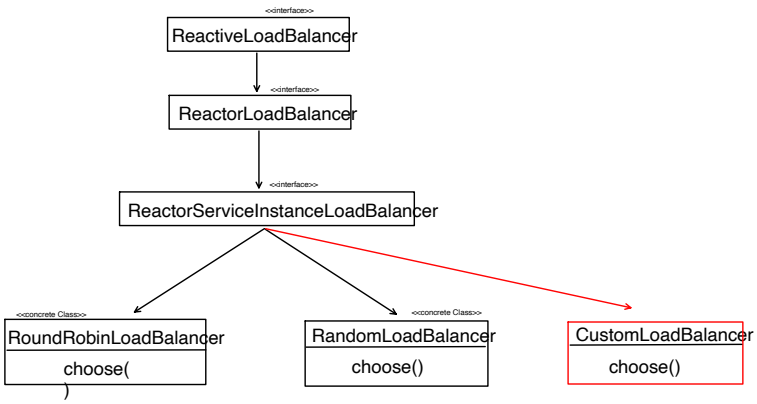
First child clients config will be loaded, then default one.

- As we already know, this *LoadBalancerClients* config is loaded at runtime for each service ID, So this *environment.getProperty(LoadBalancerClientFactory.PROPERTY_NAME)* will resolved at runtime.
- *@ConditionalOnMissingBean* is required at Global config class, because say for *"product-service"* two ReactorLoadBalancer bean should not get created one from *LoadBalancerProductClientConfig* and another from *LoadBalancerGlobalConfig*.
- So below, for "product-service" Random Load balancer algorithm will be used and for all other services "RoundRobin Algorithm will be used.

```java
@Configuration
@ConditionalOnMissingBean(ReactorLoadBalancer.class)
public class LoadBalancerGlobalConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> randomLoadBalancer(
            LoadBalancerClientFactory factory, Environment environment) {

        String serviceId = environment.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
        return new RoundRobinLoadBalancer(
                factory.getLazyProvider(serviceId, ServiceInstanceListSupplier.class),
                serviceId
        );
    }
}
```

```java
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new RandomLoadBalancer(
                factory.getLazyProvider( name: "product-service", ServiceInstanceListSupplier.class),
                serviceId: "product-service");
    }
}
```

If we want to use say our CustomLoadBalancer logic:




```java
public class MyCustomLoadBalancer implements ReactorServiceInstanceLoadBalancer {

    private final ObjectProvider<ServiceInstanceListSupplier> serviceInstanceSuppliers;
    private final String serviceId;

    public MyCustomLoadBalancer(ObjectProvider<ServiceInstanceListSupplier> serviceInstanceSuppliers,
                                String serviceId) {
        this.serviceInstanceSuppliers = serviceInstanceSuppliers;
        this.serviceId = serviceId;
    }

    @Override
    public Mono<Response<ServiceInstance>> choose(Request request) {
        return serviceInstanceSuppliers.getIfAvailable().get().next().map(instances -> {
            if (instances == null || instances.isEmpty()) {
                return new EmptyResponse();
            }
            //Custom load balancing algorithm, i am just picking first instance for demo
            return new DefaultResponse(instances.get(0));
        });
    }
}
```

```java
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new MyCustomLoadBalancer(
                factory.getLazyProvider( name: "product-service", ServiceInstanceListSupplier.class),
                serviceId: "product-service");
    }
}
```

During Service Discovery, we have already seen how FeignClient handles load balancer automatically.

So for load balancer everything is same like RestTemplate, no change at all.

Using FeignClient

So with FeignClient, load balancing is handled automatically and by the framework.

So we need to provide the Load Balancer dependency too, apart from *"spring-cloud-starter-netflix-eureka-client"*

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
```

Earlier without Service Discovery:

We need to provide the URL

```
@FeignClient(name = "product-service",
        url = "${feign.client.product-service.url}")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}
```

With Service Discovery:

No need of the URL, only the registered name of the product service is required.
and
Load Balancing will also be handled internally by the framework.

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}
```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    ProductClient productClient;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {

        String response = productClient.getProductById(id);
        System.out.println("Response from Product api call is: " + response);
    }
}
```

```
@SpringBootApplication
@EnableFeignClients
@LoadBalancerClient(name = "product-service", configuration = LoadBalancerProductClientConfig.class)
public class OrderserviceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}
```

```
@Configuration
public class LoadBalancerProductClientConfig {

    @Bean
    public ReactorLoadBalancer<ServiceInstance> productClientLoadBalancer(
            LoadBalancerClientFactory factory) {

        return new RandomLoadBalancer(
                factory.getLazyProvider( name: "product-service", ServiceInstanceListSupplier.class),
                serviceId: "product-service");
    }
}
```